3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

Daily Teaching : 2 Hours

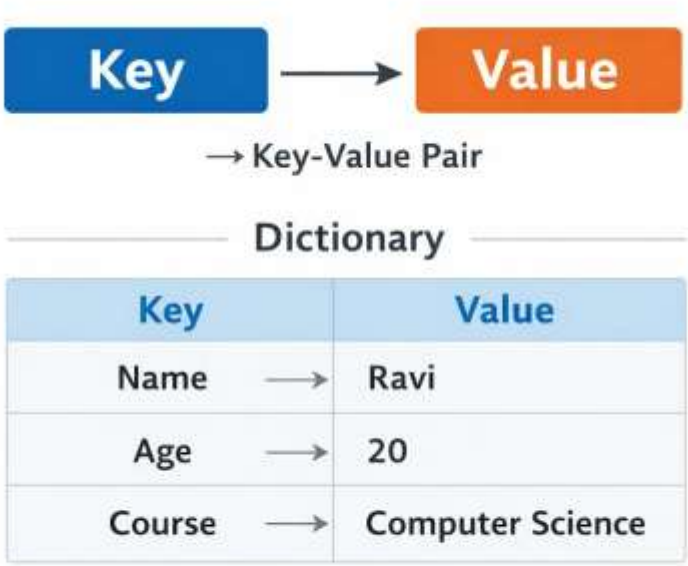
Total Classes : 100

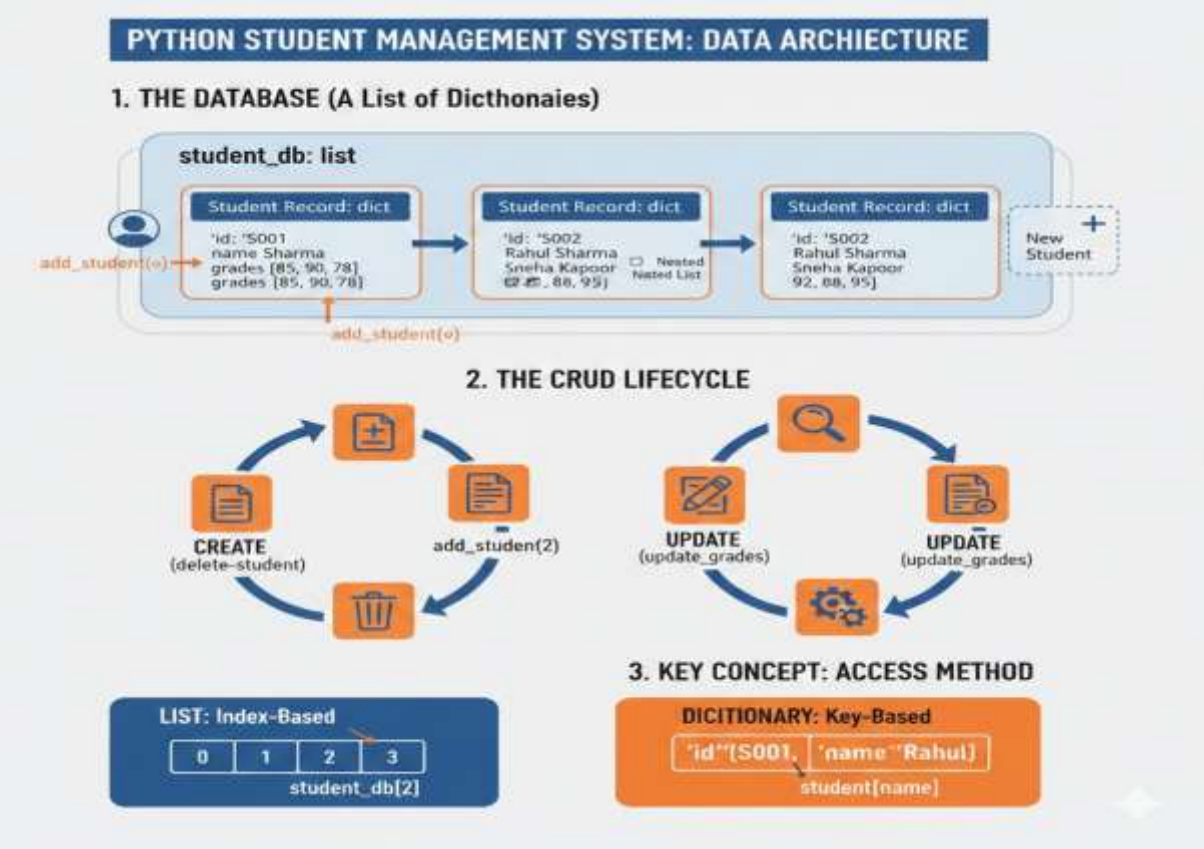
Level 4 – AI Professional Foundation(11th–12th Standard)

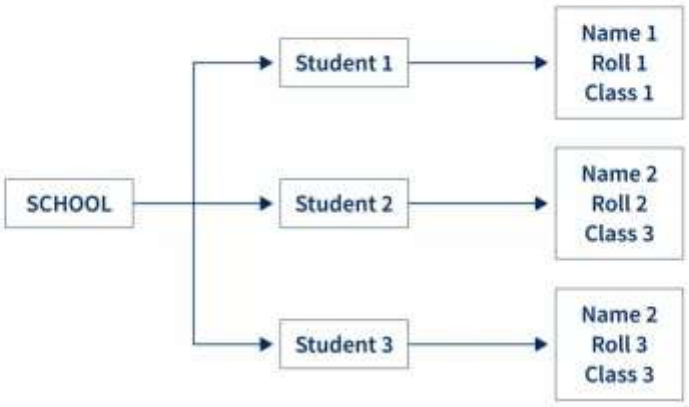
Class 7 – Dictionaries & Student Database

SOFT FOUNDATION

This lesson teaches students how to use **Python dictionaries** to store and manage student information. It also introduces the basic idea behind a **Student Database Management System**.







SCALER Topics

1. Learning Objectives

By the end of this class, students will be able to:

- Understand what a dictionary is in Python.
- Create and access dictionary elements.
- Add, modify and delete dictionary data.
- Use dictionary methods.
- Store student information using dictionaries.
- Create a simple student database.
- Search, update and delete student records.
- Work with **nested dictionaries**.
- Write a menu-driven Student Database program.

PART A – THEORY

2. What is a Dictionary?

A **dictionary** is a Python data structure that stores data in **key-value pairs**.

Example

```
student = {
   "name": "Rahul",
   "age": 17,
   "class": 12,
   "marks": 85
}
```

Here:

Key	Value
name	Rahul
age	17

3DWEBSOFT FOUNDATION

Key	Value
class	12
marks	85

Think of a dictionary like a real student record:

Student ID → Student Information

101 → Rahul, Class 12, 85 marks

102 → Priya, Class 11, 91 marks

3. Dictionary Syntax

```
dictionary_name = {
   "key1": value1,
   "key2": value2,
   "key3": value3
}
```

Example:

```
student = {
   "name": "Anjali",
   "age": 16,
   "marks": 92
}
```

4. Important Properties of Dictionaries

Python dictionaries:

1. Store data as **key-value pairs**.
2. Keys must be unique.

3DWEBSOFT FOUNDATION

3. Values can be of different data types.
4. Dictionaries are mutable.
5. Dictionaries preserve insertion order in modern Python.
6. Values can be accessed using their keys.

5. Creating a Dictionary

```
student = {
    "name": "Arjun",
    "age": 17,
    "course": "Python",
    "marks": 88
}

print(student)
```

Output

```
{'name': 'Arjun', 'age': 17, 'course': 'Python', 'marks': 88}
```

6. Accessing Dictionary Values

We can access a value using its key.

```
student = {
    "name": "Arjun",
    "age": 17,
    "marks": 88
}
```

3DWEBSOFT FOUNDATION

```
print(student["name"])
print(student["marks"])
```

Output

```
Arjun
88
```

Important

student["name"]

means:

Find the value associated with the "name" key.

7. Using get()

The get() method is a safer way to access dictionary values.

```
student = {
   "name": "Arjun",
   "marks": 88
}

print(student.get("name"))
print(student.get("age"))
```

Output:

```
Arjun
None
```

We can also provide a default value:

```
print(student.get("age", "Not Available"))
```

Output:

3DWEBSOFT FOUNDATION

Not Available

8. Adding New Data

We can add a new key-value pair.

```
student = {
   "name": "Arjun",
   "marks": 88
}

student["age"] = 17

print(student)
```

Output:

```
{'name': 'Arjun', 'marks': 88, 'age': 17}
```

9. Updating Dictionary Data

Existing values can be changed.

```
student = {
   "name": "Arjun",
   "marks": 88
}

student["marks"] = 95

print(student)
```

3DWEBSOFT FOUNDATION

Output:

{'name': 'Arjun', 'marks': 95}

10. Deleting Dictionary Data

Using del

```
student = {
   "name": "Arjun",
   "age": 17,
   "marks": 88
}

del student["age"]

print(student)
```

Using pop()

```
student.pop("marks")

print(student)
```

pop() removes the specified key and returns its value.

Using clear()

```
student.clear()

print(student)
```

3DWEBSOFT FOUNDATION

Output:

{}

11. Important Dictionary Methods

Method	Purpose
get()	Gets a value
keys()	Returns all keys
values()	Returns all values
items()	Returns key-value pairs
update()	Updates dictionary
pop()	Removes a key
popitem()	Removes last inserted item
clear()	Removes all items
copy()	Creates a copy

12. keys()

```
student = {
   "name": "Rahul",
   "age": 17,
   "marks": 90
}

print(student.keys())
```

3DWEBSOFT FOUNDATION

13. values()

print(student.values())

This returns all values.

14. items()

print(student.items())

It returns key-value pairs.

We can use a loop:

```
for key, value in student.items():
    print(key, ":", value)
```

Output:

name : Rahul

age : 17

marks : 90

15. Checking Whether a Key Exists

Use the in operator.

```
student = {
    "name": "Rahul",
    "marks": 90
}

if "marks" in student:
    print("Marks are available")
```

Output:

3DWEBSOFT FOUNDATION

Marks are available

16. Dictionary with Different Data Types

A dictionary can contain strings, integers, lists, etc.

```
student = {
   "name": "Sneha",
   "age": 17,
   "marks": [85, 90, 92],
   "passed": True
}

print(student)
```

PART B – STUDENT DATABASE

17. What is a Student Database?

A **student database** is a structured collection of information about students.

It may contain:

- Student ID
- Student name
- Age
- Class
- Section
- Phone number
- Subjects
- Marks

3DWEBSOFT FOUNDATION

- Attendance

Student

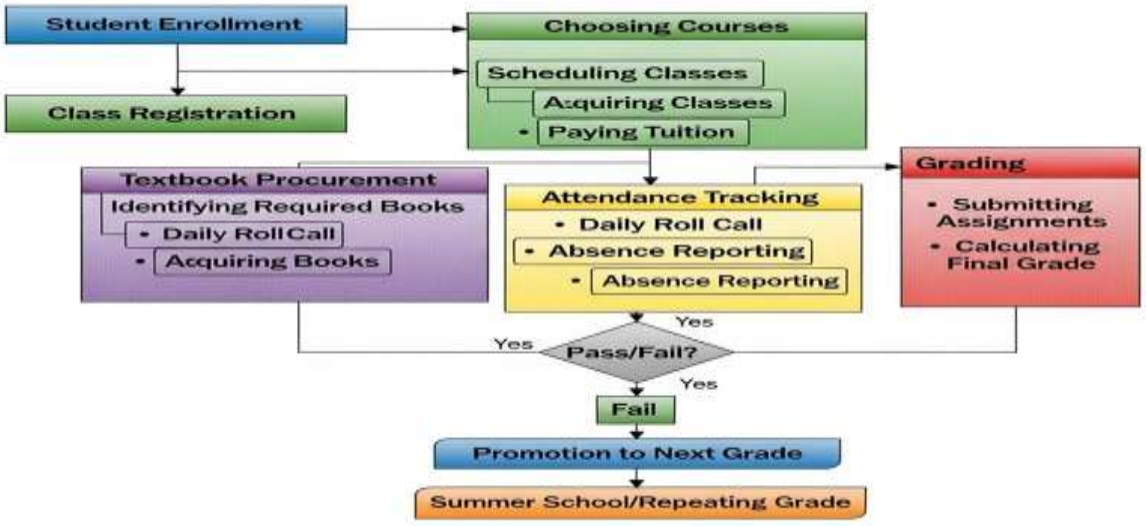
Id	Name
1	Abe
2	Bob
3	Chris

Course

Id	Name
1	Math
2	Computers
3	Science

Marks

StudentId	CourseId	Marks
1	1	90
1	2	80
2	1	70
2	3	91
3	3	81
3	2	92





18. Why Use Dictionaries for a Student Database?

Dictionaries are useful because information can be identified using meaningful keys.

Instead of:

student[0]

student[1]

student[2]

we can write:

student["name"]

student["class"]

student["marks"]

3DWEBSOFT

This makes programs easier to understand.

19. One Student Record

```
student = {
   "id": 101,
   "name": "Rahul",
   "class": 12,
   "section": "A",
   "marks": 89
}


print(student)
```

Access individual information:

```
print(student["name"])
print(student["marks"])
```

20. Multiple Students – Nested Dictionary

When storing multiple students, we can use a dictionary inside another dictionary.

```
students = {
   101: {
      "name": "Rahul",
      "class": 12,
      "marks": 89
   },
```

3DWEBSOFT FOUNDATION

```
    102: {
        "name": "Priya",
        "class": 11,
        "marks": 94
    }
}
```

Here:

```
students
  |
  ├── 101
  |     ├── name → Rahul
  |     ├── class → 12
  |     └── marks → 89
  |
  └── 102
        ├── name → Priya
        ├── class → 11
        └── marks → 94
```

21. Accessing a Student Record

```
print(students[101])
```

To access the student's name:

```
print(students[101]["name"])
```

Output:

Rahul

3DWEBSOFT FOUNDATION

To access marks:

```
print(students[101]["marks"])
```

22. Adding a Student

```
students[103] = {
    "name": "Aisha",
    "class": 12,
    "marks": 91
}

print(students)
```

23. Updating a Student

```
students[101]["marks"] = 95
```

Now Rahul's marks are updated to 95.

24. Deleting a Student

```
del students[102]
```

Student ID 102 is removed.

25. Displaying All Students

```
for student_id, details in students.items():
    print("Student ID:", student_id)
    print("Name:", details["name"])
    print("Class:", details["class"])
```

3DWEBSOFT FOUNDATION

```
    print("Marks:", details["marks"])
    print("----------------")
```

26. Searching for a Student

```
student_id = int(input("Enter Student ID: "))

if student_id in students:
    print(students[student_id])
else:
    print("Student not found")
```

PART C – PRACTICAL PROGRAM

27. Simple Student Database

Program

```
students = {
    101: {
        "name": "Rahul",
        "class": 12,
        "marks": 89
    },

    102: {
        "name": "Priya",
        "class": 11,
        "marks": 94
```

3DWEBSOFT FOUNDATION

```
    }
}

print("Student Database")

for student_id, details in students.items():
    print("ID:", student_id)
    print("Name:", details["name"])
    print("Class:", details["class"])
    print("Marks:", details["marks"])
    print("--------------------")
```

28. Menu-Driven Student Database

This is the main practical activity for the lesson.

Features

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Complete Program

```
students = {}

while True:
```

3DWEBSOFT FOUNDATION

```
print("\n===== STUDENT DATABASE =====")
print("1. Add Student")
print("2. View Students")
print("3. Search Student")
print("4. Update Student")
print("5. Delete Student")
print("6. Exit")

choice = input("Enter your choice: ")

# Add Student
if choice == "1":

    student_id = int(input("Enter Student ID: "))

    name = input("Enter Student Name: ")
    student_class = input("Enter Class: ")
    marks = float(input("Enter Marks: "))

    students[student_id] = {
        "name": name,
        "class": student_class,
        "marks": marks
    }
```

3DWEBSOFT FOUNDATION

```
    print("Student added successfully!")

# View Students
elif choice == "2":

    if not students:
        print("No students available.")

    else:
        for student_id, details in students.items():

            print("\nStudent ID:", student_id)
            print("Name:", details["name"])
            print("Class:", details["class"])
            print("Marks:", details["marks"])

# Search Student
elif choice == "3":

    student_id = int(input("Enter Student ID: "))

    if student_id in students:
        print("\nStudent Found")
        print("Name:", students[student_id]["name"])
```

3DWEBSOFTFOUNDATION

```
        print("Class:", students[student_id]["class"])
        print("Marks:", students[student_id]["marks"])

    else:
        print("Student not found.")

# Update Student
elif choice == "4":

    student_id = int(input("Enter Student ID: "))

    if student_id in students:

        name = input("Enter new name: ")
        marks = float(input("Enter new marks: "))

        students[student_id]["name"] = name
        students[student_id]["marks"] = marks

        print("Student updated successfully!")

    else:
        print("Student not found.")

# Delete Student
```

3DWEBSOFT FOUNDATION

```
    elif choice == "5":

        student_id = int(input("Enter Student ID: "))

        if student_id in students:
            del students[student_id]
            print("Student deleted successfully!")

        else:
            print("Student not found.")

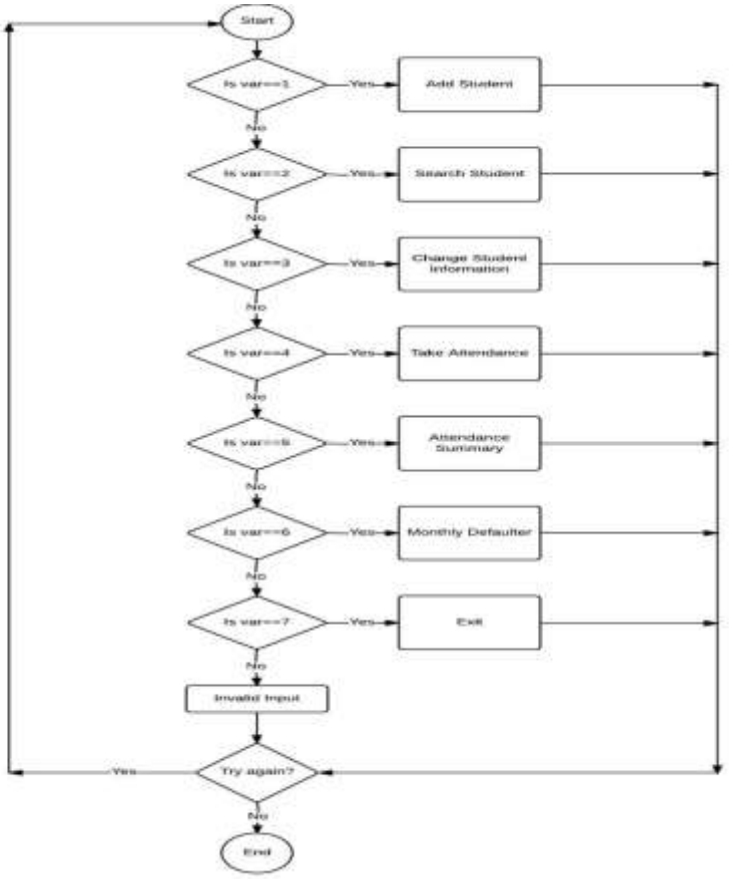
    # Exit
    elif choice == "6":

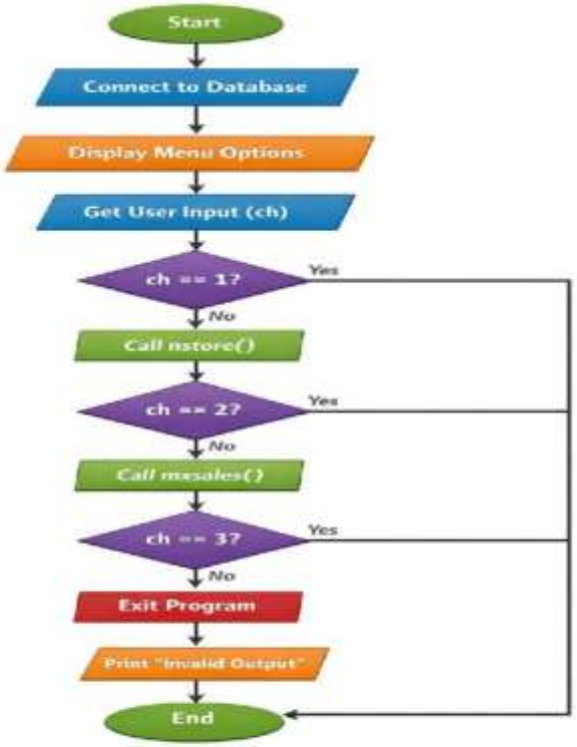
        print("Thank you!")
        break

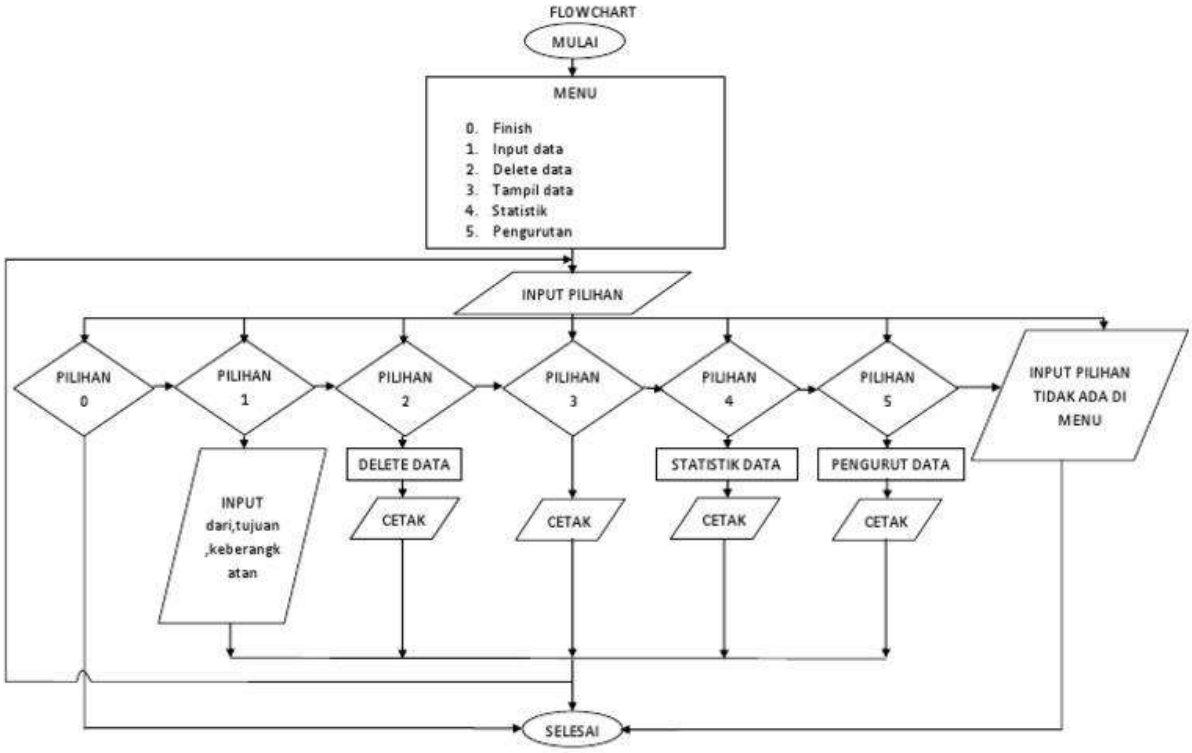
    else:
        print("Invalid choice.")
```

29. Program Flow

3DWEBSOFT FOUNDATION


Start
Is var==1
Yes
Add Student
No
Is var==2
Yes
Search Student
No
Is var==3
Yes
Change Student Information
No
Is var==4
Yes
Take Attendance
No
Is var==5
Yes
Attendance Summary
No
Is var==6
Yes
Monthly Defaulter
No
Is var==7
Yes
Exit
No
Invalid Input
Try again?
Yes
No
End


Start
Connect to Database
Display Menu Options
Get User Input (ch)
ch == 1?
Yes
No
Call nstore()
ch == 2?
Yes
No
Call mxsales()
ch == 3?
Yes
No
Exit Program
Print "Invalid Output"
End



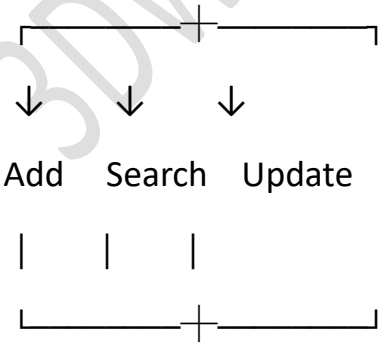
The program follows this flow:

START

↓

Display Main Menu

↓



Add Search Update

↓

Delete

3DWEBSOFT

↓

View Students

↓

Exit?

/ \

No Yes

↓ ↓

Menu END

30. CRUD Concept

The student database demonstrates **CRUD** operations.

CRUD	**Meaning**	**Student Database**
C	Create	Add Student
R	Read	View/Search Student
U	Update	Change Student Data
D	Delete	Remove Student

This is an important concept because real-world applications use CRUD extensively.

3DWEBSOFT FOUNDATION

Practical Session

Practical Activity 1 – Student Marks

Create a dictionary:

student = {

```
    "name": "Ravi",
    "maths": 85,
    "physics": 78,
    "computer": 92
}
Calculate the total:
total = (
    student["maths"]
    + student["physics"]
    + student["computer"]
)

print("Total:", total)
Calculate average:
average = total / 3

print("Average:", average)
```

Practical Activity 2 – Grade Calculator

```
student = {
    "name": "Ravi",
    "marks": 86
}

marks = student["marks"]
```

3DWEBSOFT FOUNDATION

```
if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "D"

print("Student:", student["name"])
print("Grade:", grade)
```

Practical Activity 3 – Student Search

Create five students:

```
students = {
    101: {"name": "Ravi", "marks": 85},
    102: {"name": "Anu", "marks": 92},
    103: {"name": "Kiran", "marks": 78},
    104: {"name": "Sara", "marks": 95},
    105: {"name": "Aman", "marks": 88}
}
```

Ask the user for an ID and display the student.

3DWEBSOFT FOUNDATION

Practical Activity 4 – Find Highest Marks

```
students = {
    101: {"name": "Ravi", "marks": 85},
    102: {"name": "Anu", "marks": 92},
    103: {"name": "Kiran", "marks": 78},
    104: {"name": "Sara", "marks": 95}
}

highest_id = max(
    students,
    key=lambda x: students[x]["marks"]
)

print("Top Student:", students[highest_id]["name"])
print("Marks:", students[highest_id]["marks"])
```

Common Errors

Error 1 – Using a wrong key

```
student = {"name": "Ravi"}

print(student["age"])
```

This produces a KeyError.

Use:

```
print(student.get("age", "Not Available"))
```

3DWEBSOFT FOUNDATION

Error 2 – Duplicate keys

```
student = {
   "name": "Ravi",
   "name": "Anu"
}
```

The second "name" replaces the first one.

Error 3 – Forgetting quotes around string keys

Incorrect:

```
student = {
   name: "Ravi"
}
```

Correct:

```
student = {
   "name": "Ravi"
}
```

3DWEBSOFT FOUNDATION

36. Dictionary vs List

Feature	List	Dictionary
Data access	Index	Key
Example	student[0]	student["name"]
Structure	Ordered values	Key-value pairs
Best for	Collection of items	Named information

Feature	List	Dictionary
Example use	Marks list	Student record

37. Classroom Activity

Ask students to create their own student record.

```
student = {
    "name": "Your Name",
    "age": 17,
    "class": 12,
    "section": "A",
    "school": "ABC School",
    "marks": 90
}
```

Then ask them to:

1. Print the name.
2. Print the marks.
3. Change the marks.
4. Add a phone number.
5. Delete the age.
6. Display all keys.
7. Display all values.

38. Mini Project

Student Database Management System

3DWEBSOFT FOUNDATION

Students should build a program containing:

==========

STUDENT DATABASE

==========

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Suggested student record

```
{
   "name": "Ayesha",
   "age": 17,
   "class": 12,
   "section": "A",
   "marks": 91
}
```

Skills Practiced

- Dictionary
- Nested dictionary
- Variables
- Input/output
- if-elif-else

3DWEBSOFT FOUNDATION

- while loop
- for loop
- Functions
- CRUD operations
- Searching
- Updating
- Deleting

39. Quick Revision

Remember:

Dictionary = Key + Value

```
student = {
   "name": "Ravi",
   "marks": 90
}
```

Access:

```
student["name"]
```

Add:

```
student["age"] = 17
```

Update:

```
student["marks"] = 95
```

Delete:

```
del student["age"]
```

Check:

```
if "name" in student:
```

3DWEBSOFT FOUNDATION

```
    print("Found")
Loop:
for key, value in student.items():
    print(key, value)
```

40. Homework

Basic

1. What is a dictionary?
2. What is a key-value pair?
3. Write the syntax of a dictionary.
4. How do you add a new item?
5. What is the difference between get() and []?
6. Explain keys(), values() and items().

Programming

Q1. Create a dictionary containing your name, age, class and marks.

Q2. Write a program to add a new subject and marks.

Q3. Write a program to calculate the average marks.

Q4. Create a database containing 5 students.

Q5. Build a menu-driven Student Database Management System using dictionaries.

3DWEBSOFT FOUNDATION

Teacher's Assessment

Skill	Marks
Dictionary creation	2

Skill	Marks
Dictionary methods	2
Nested dictionaries	2
Student search	2
CRUD implementation	4
Program logic	4
Output/presentation	2
Total	**18**

3DWEBSOFT FOUNDATION